

http://algs4.cs.princeton.edu

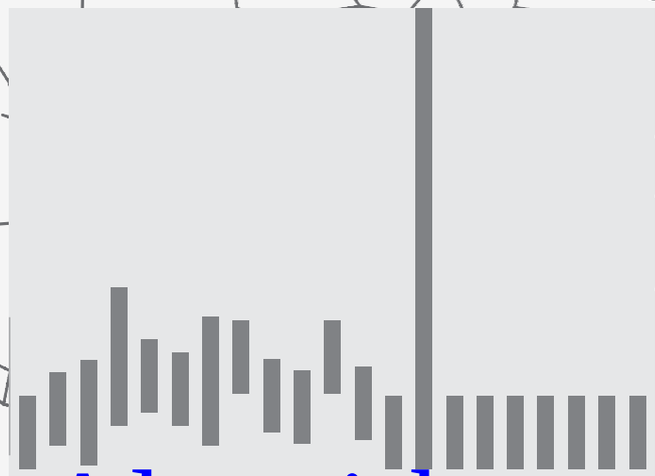
5.2 TRIES

- *R-way tries*
- *ternary search tries*
- *character-based operations*

String symbol table basic API

String symbol table. Symbol table i specializuar në string çelësa.

public class StringST<Value>	
StringST()	*create një symbol table të zbrazët*
void put(String key, Value val)	*put çelës-vlerë çiftin në symbol table*
Value get(String key)	*return vlerën çiftuar me çelësin e dhënë*
void delete(String key)	*delete çelësin dhe vlerën përkatëse*
⋮	



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

5.2 TRIES

- *R-way tries*
- *ternary search tries*
- *character-based operations*

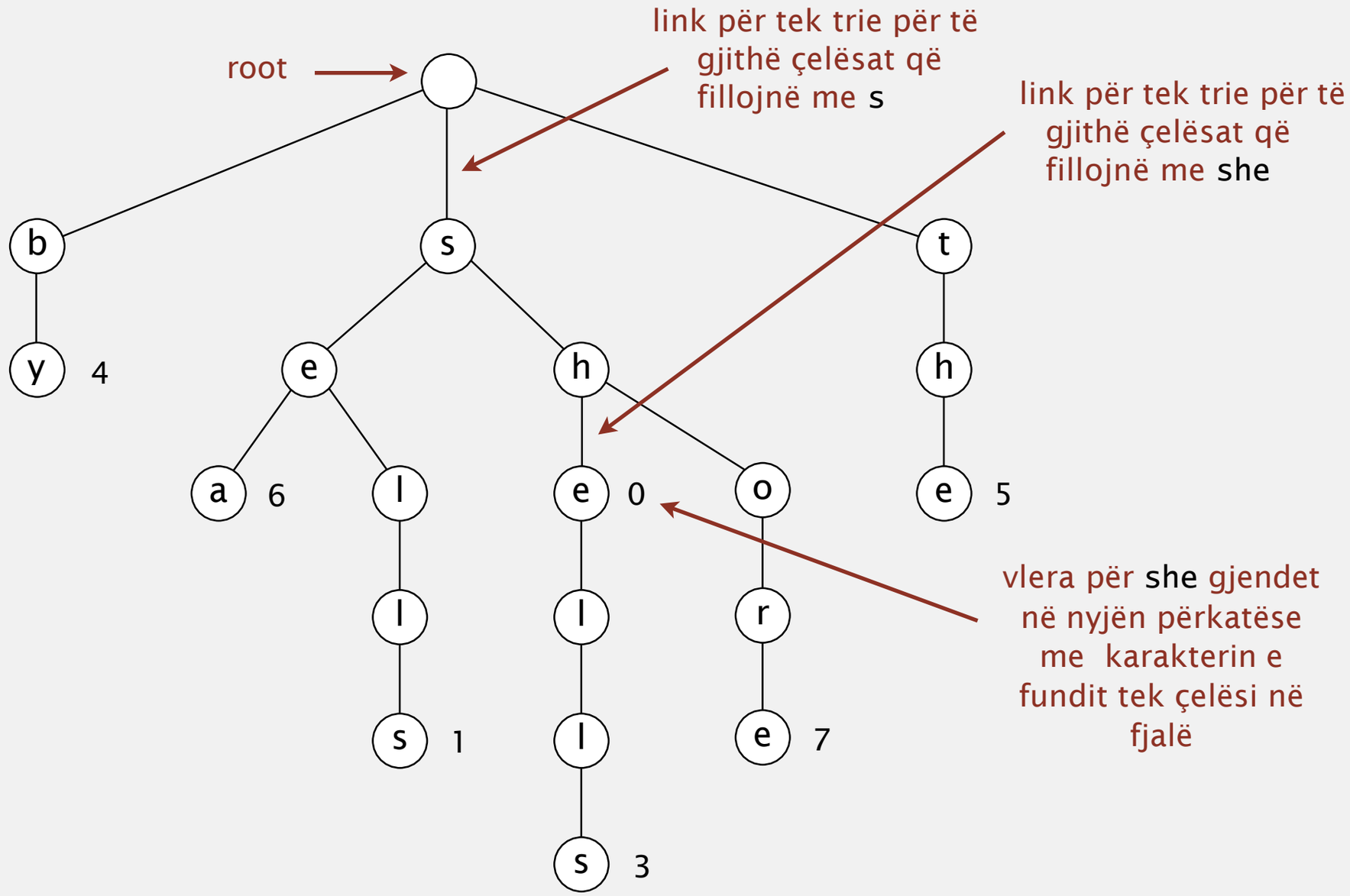
Tries



Tries

Tries. [from retrieval, but pronounced "try"]

- Ruaj karakteret në nyje (jo në çelësa).
- Secila nyje ka R fëmijë, një për çdo karakter të mundshëm.

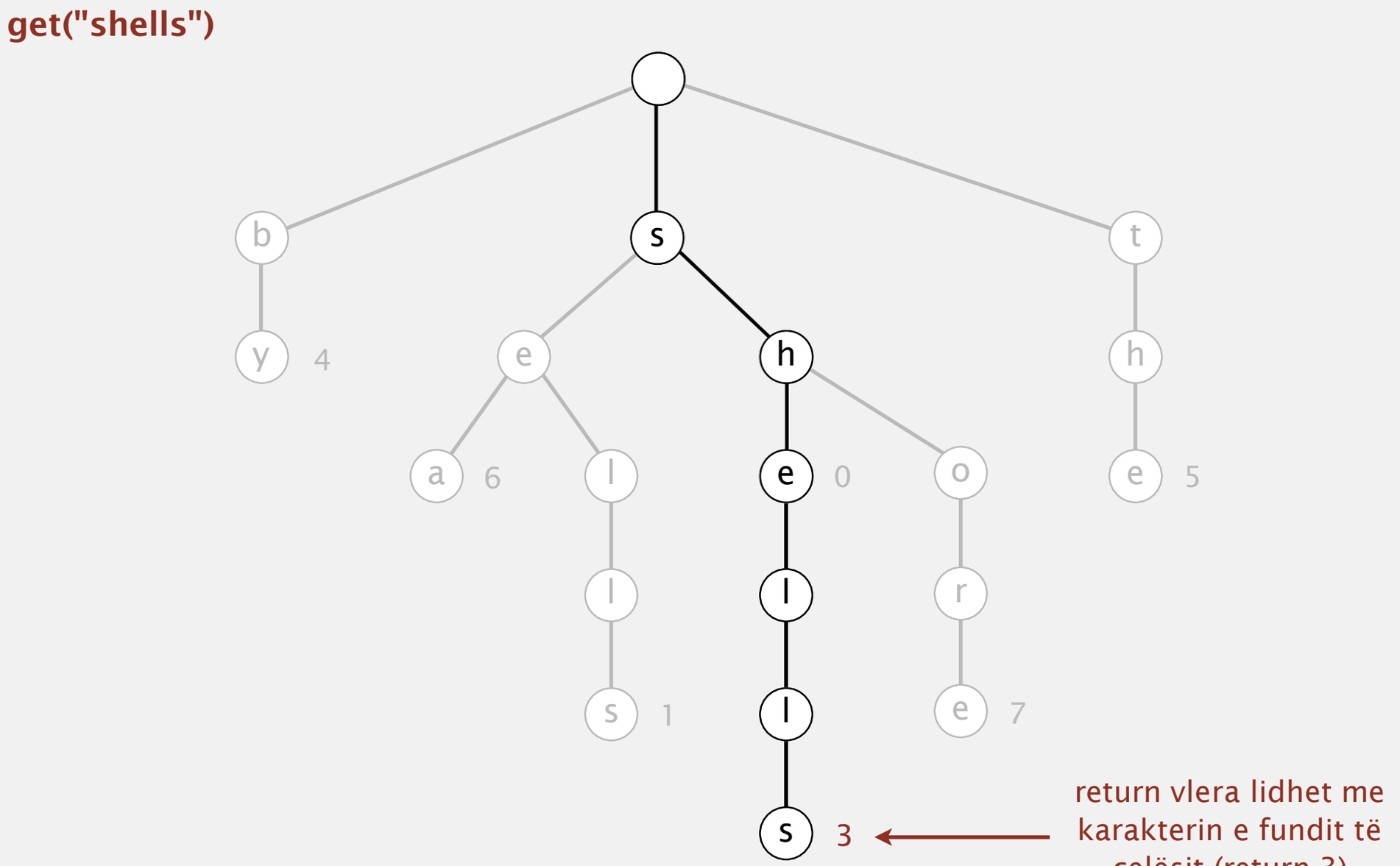


çelësi	vlera
by	4
sea	6
sells	1
she	0
shells	3
shore	7
the	5

Search në një trie

Të përcillet linku përkatës për secilin karakter në çelës

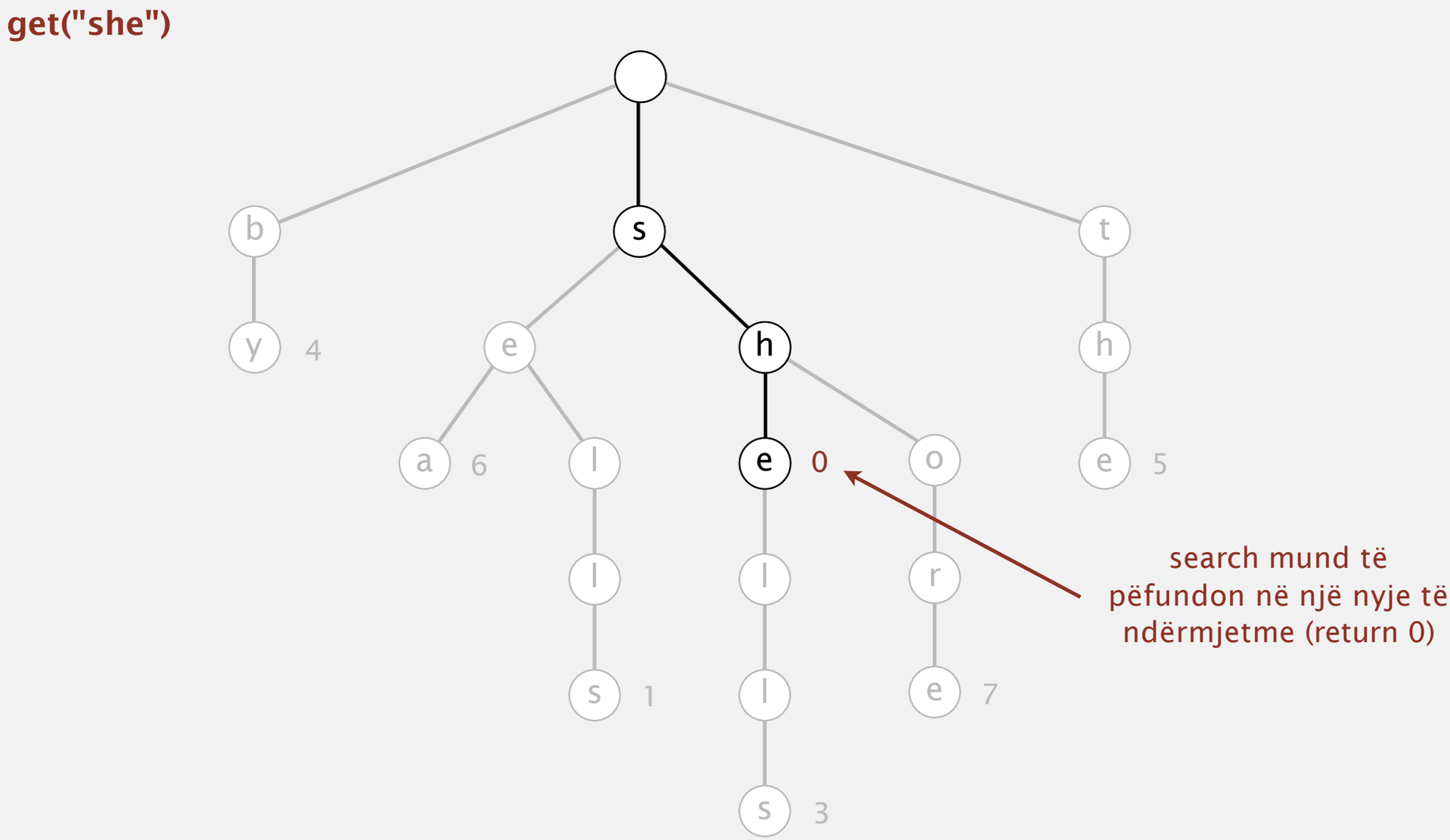
- Search hit: kur nyja ku search përfundon ka vlerën jo-null.
- Search miss: kur arrin null link ose nyja ku search përfundon ka vlerën null.



Search në një trie

Të përcillet linku përkatës për secilin karakter në çelës

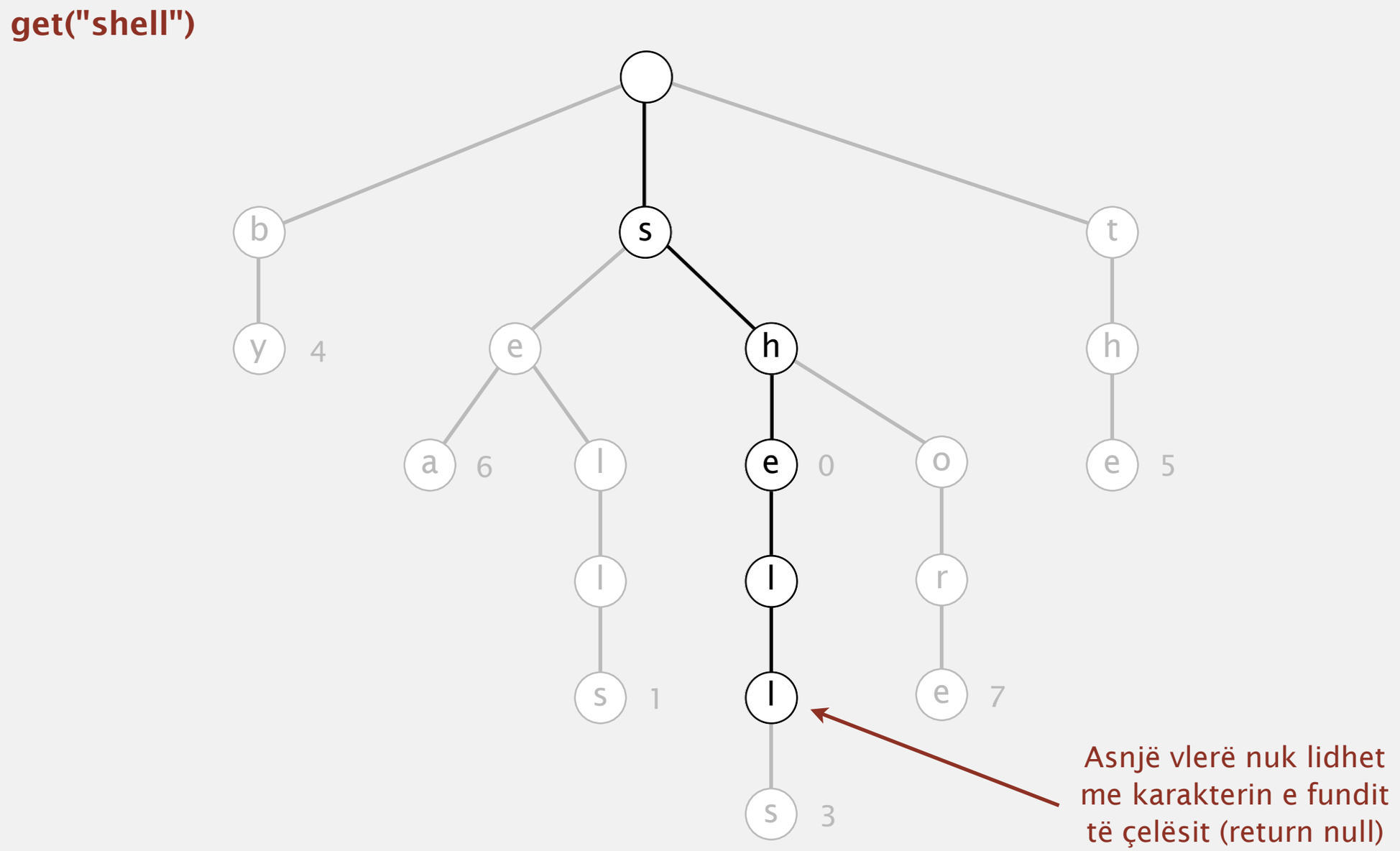
- Search hit: kur nyja ku search përfundon ka vlerën jo-null.
- Search miss: kur arrin null link ose nyja ku search përfundon ka vlerën null.



Search në një trie

Të përcillet linku përkatës për secilin karakter në çelës

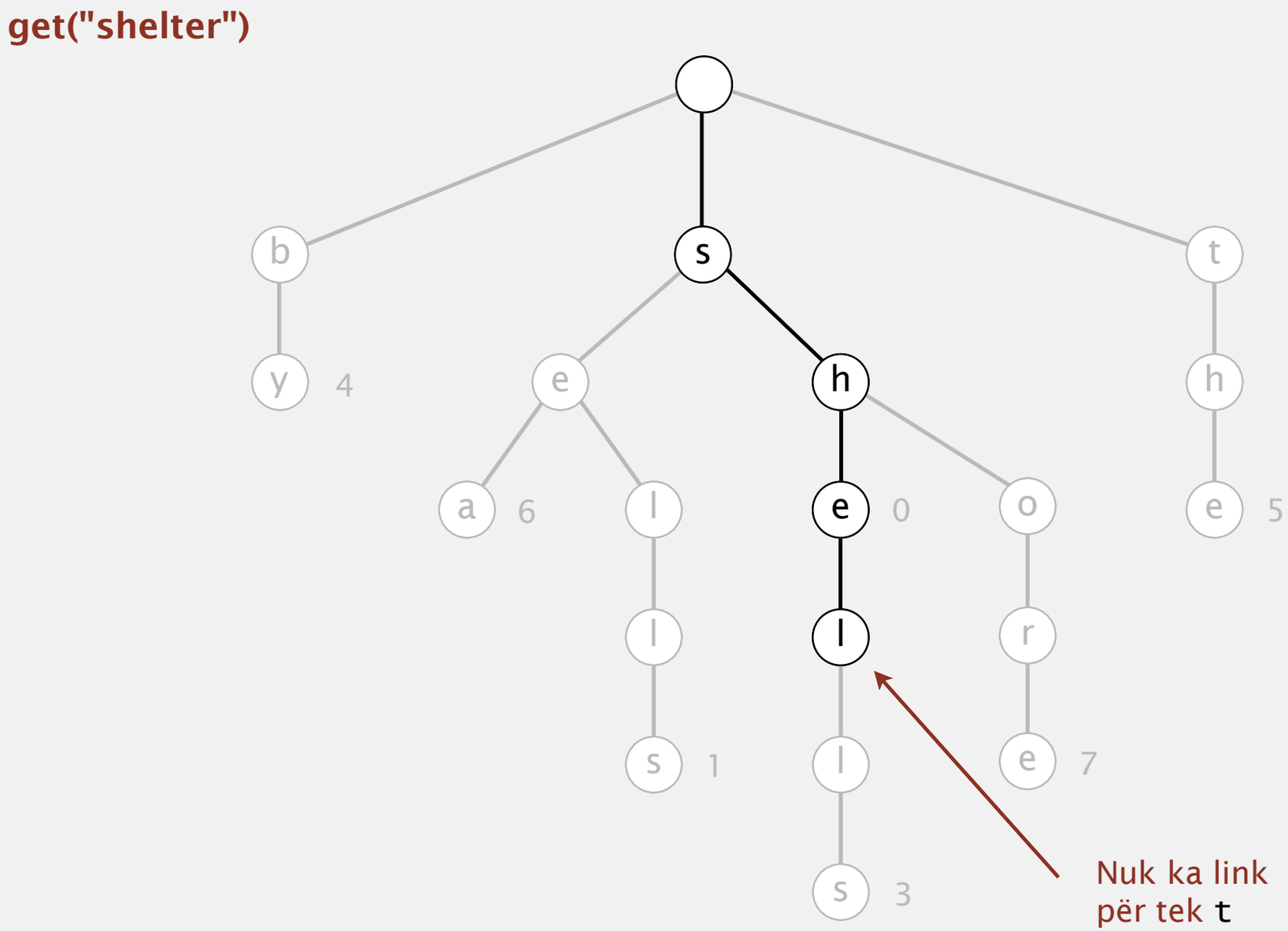
- Search hit: kur nyja ku search përfundon ka vlerën jo-null.
- Search miss: kur arrin null link ose nyja ku search përfundon ka vlerën null



Search në një trie

Të përcillet linku përkatës për secilin karakter në çelës

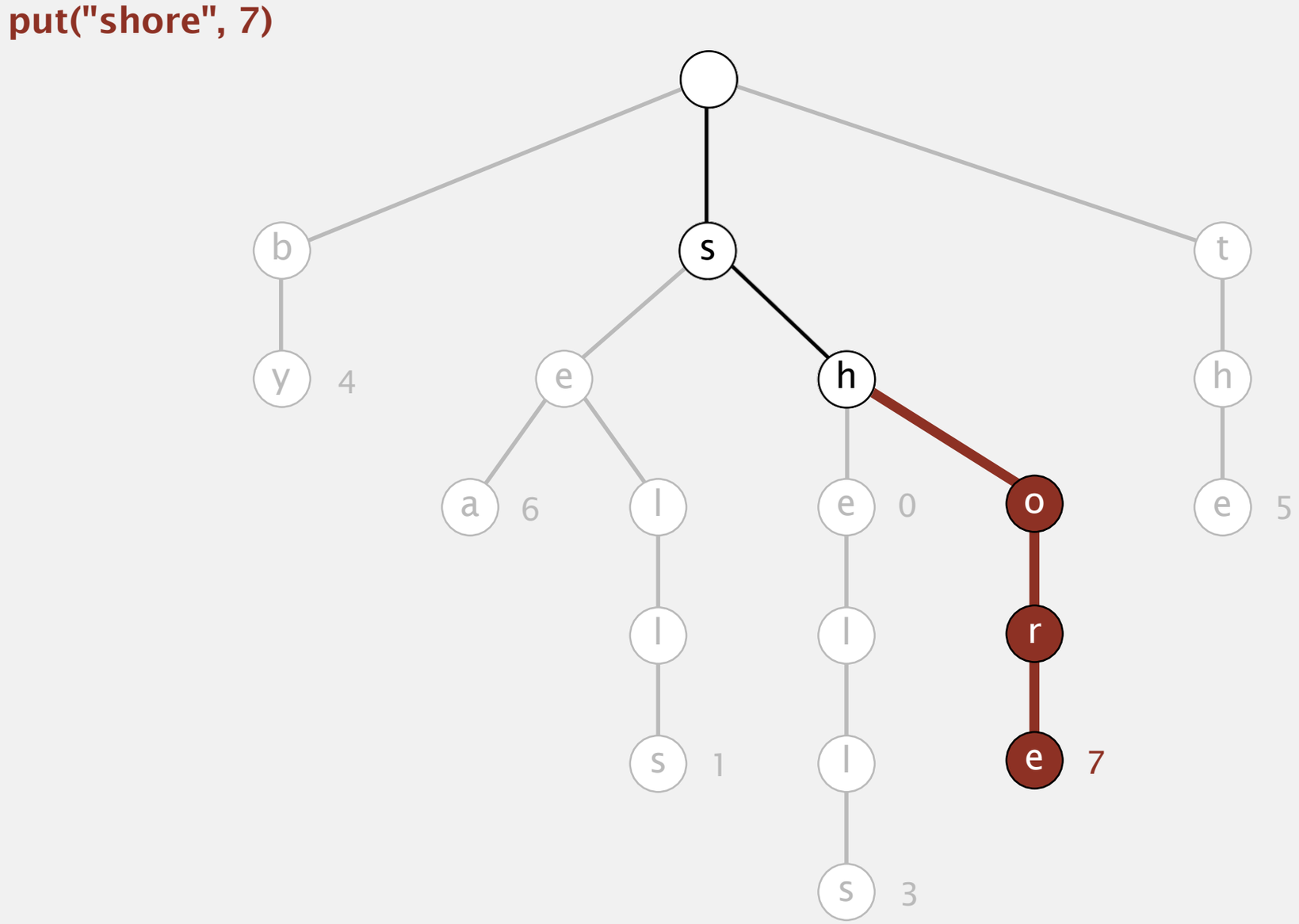
- Search hit: kur nyja ku search përfundon ka vlerën jo-null.
- Search miss: kur arrin null link ose nyja ku search përfundon ka vlerën null



Insertion në një trie

Të përcillet linku përkatës për secilin karakter në çelës

- Kur arrin tek një null link: Krijo një nyje të re.
- Kur arrin tek karakteri i fundit i çelësit: set vlerën në atë nyje.

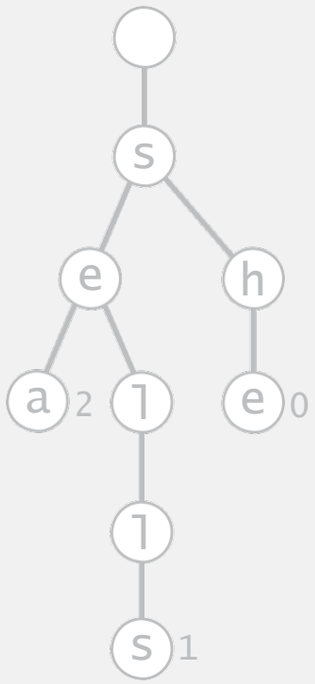


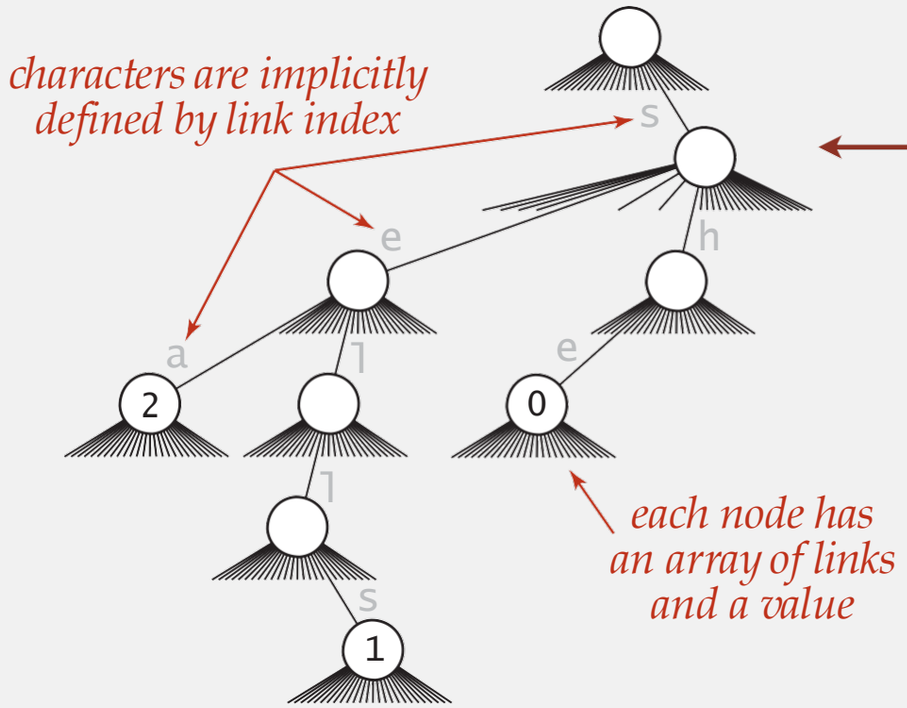
Trie representation: Java implementation

Node. A value, plus references to R nodes.

```
private static class Node
{
   private Object value;
   private Node[] next = new Node[R];
}
```

no generic array creation in Java





neither keys nor characters are explicitly stored

each node has an array of links and a value

R-way trie: Java implementation

```
public class TrieST<Value>
{
   private static final int R = 256;      <-- extended ASCII
   private Node root = new Node();

   private static class Node
   {  /* see previous slide */  }

   public void put(String key, Value val)
   {  root = put(root, key, val, 0);  }

   private Node put(Node x, String key, Value val, int d)
   {
      if (x == null) x = new Node();
      if (d == key.length()) { x.val = val; return x; }
      char c = key.charAt(d);
      x.next[c] = put(x.next[c], key, val, d+1);
      return x;
   }

   ⋮
```

R-way trie: Java implementation (continued)

```
   ⋮
public boolean contains(String key)
{  return get(key) != null;  }

public Value get(String key)
{
   Node x = get(root, key, 0);
   if (x == null) return null;
   return (Value) x.val;
}

   private Node get(Node x, String key, int d)
   {
      if (x == null) return null;
      if (d == key.length()) return x;
      char c = key.charAt(d);
      return get(x.next[c], key, d+1);
   }

}
```

cast needed

Trie - performanca

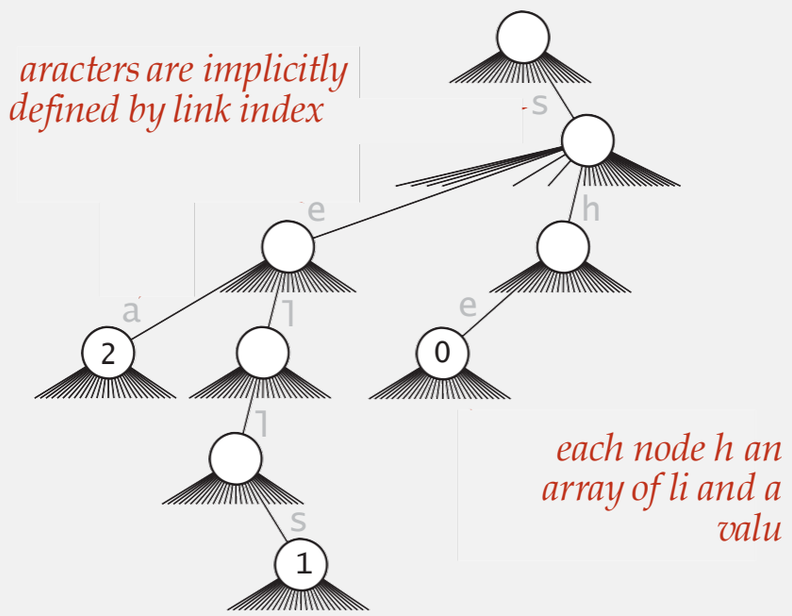
Search hit. Duhet të kontrollohen të gjitha karakteret për barazi.

Search miss.

- Mund të ketë mospërputhje në karakterin e parë.
- Rasti tipik: kontrollohen vetëm disa karaktere (sublineare).

Hapësira. R null links tek secili leaf (gjeth).

(por sublinear hapësira është e mundshme nëse ka shumë string të shkurtë që kanë prefix të përbashkët)

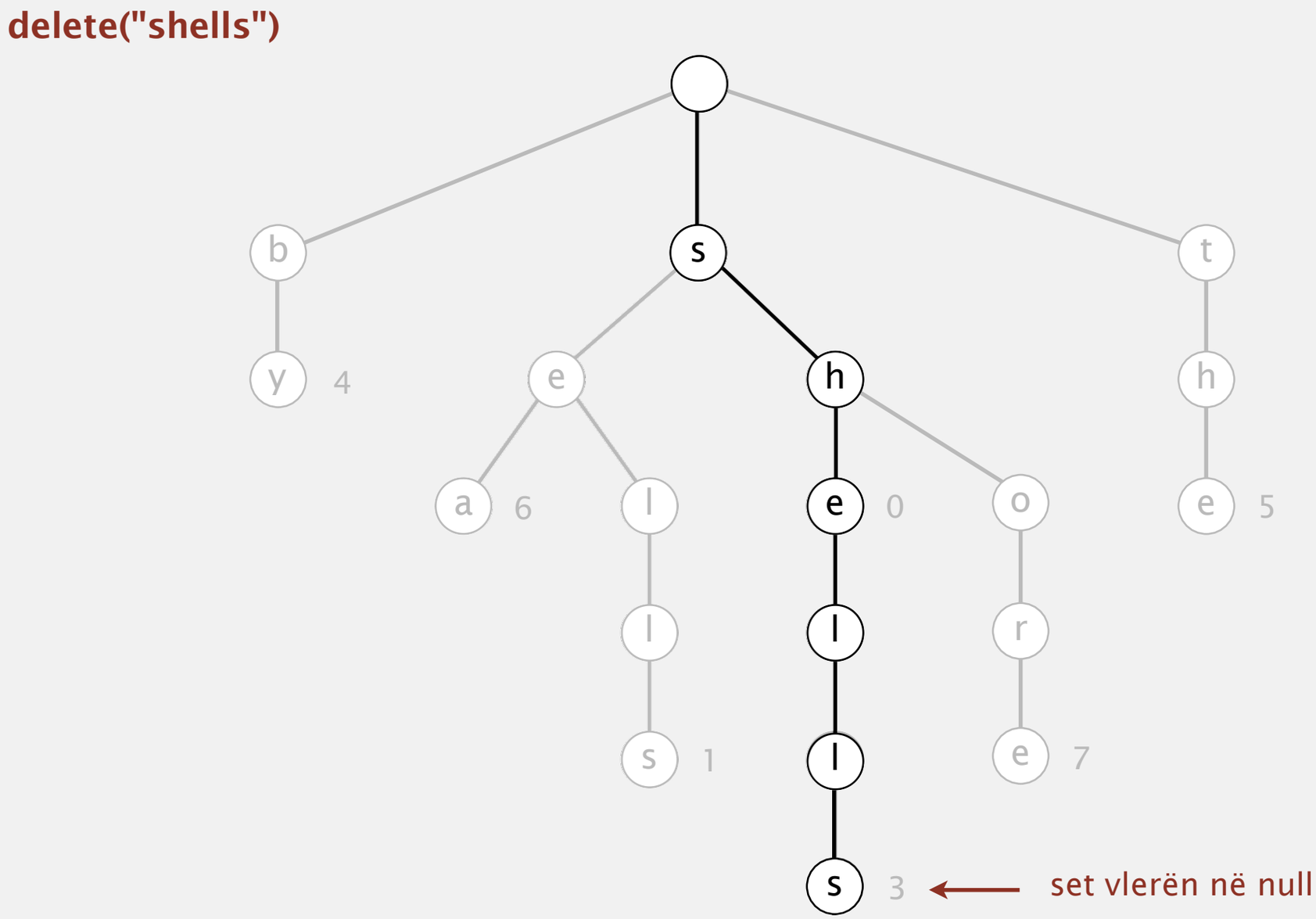


Përfundim. Search hit i shpejtë dhe search miss edhe më i shpejtë, mirëpo shpenzon hapësirë.

Delete në një R-way trie

Për të delete (fshirë) një çift çelës-vlerë:

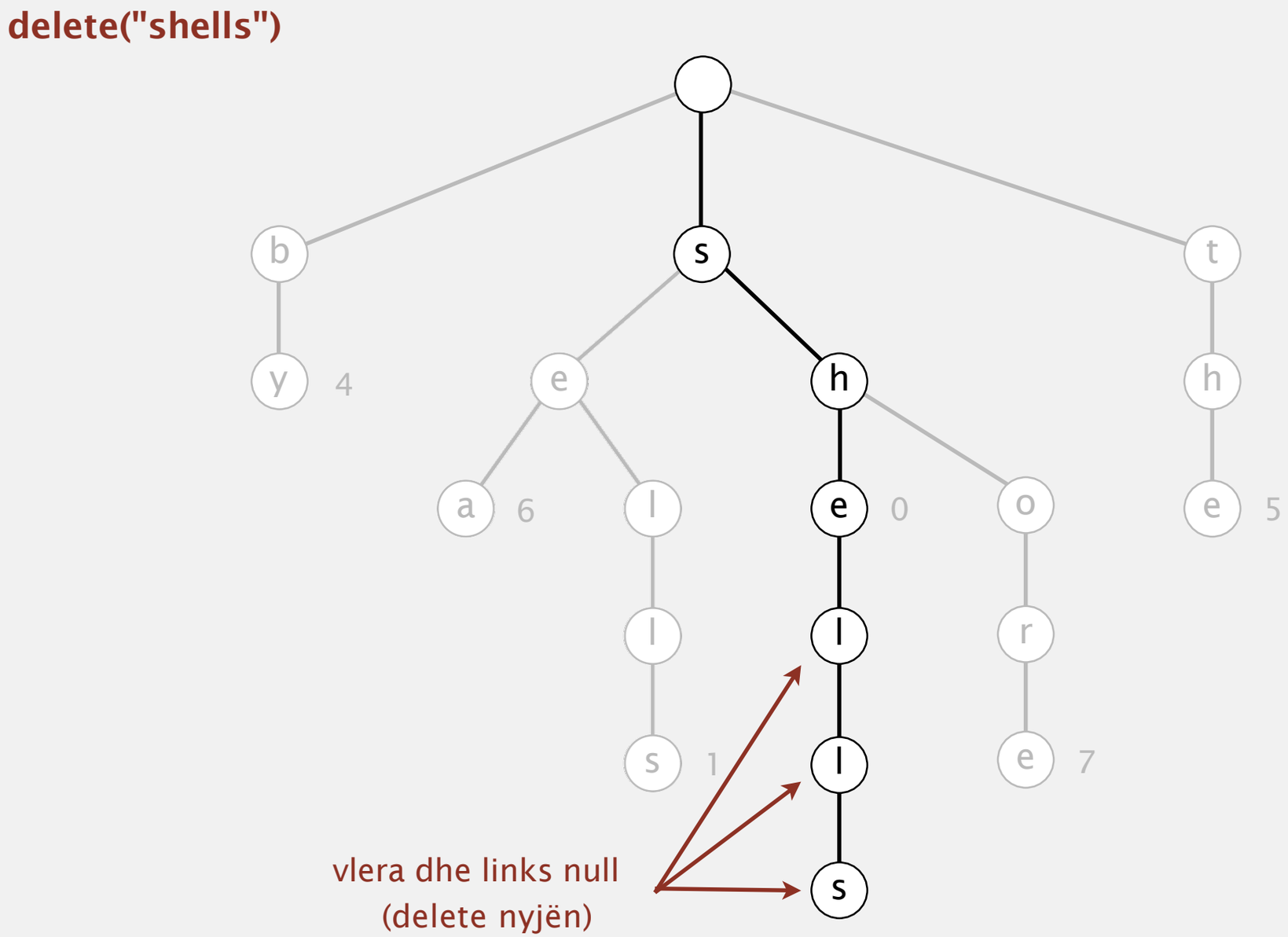
- Të gjindet nyja përkatëse me çelësin dhe të vendoset set() vlera në null.
- Nëse nyja ka vlerën null dhe të gjitha links null, remove atë nyje.

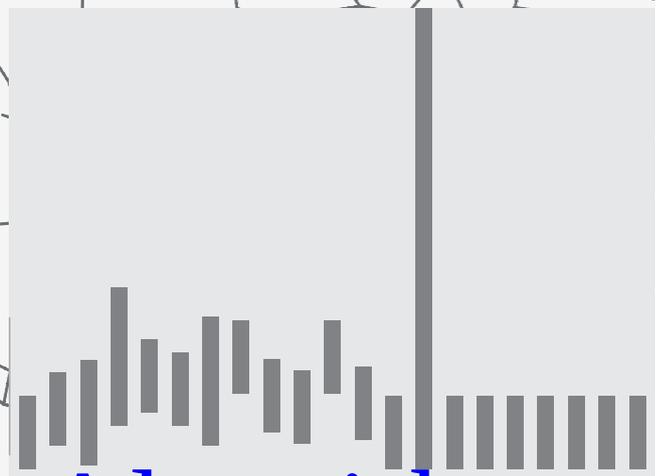


Delete në një R-way trie

Për të delete (fshirë) një çift çelës-vlerë:

- Të gjindet nyja përkatëse me çelësin dhe të vendoset set() vlera në null.
- Nëse nyja ka vlerën null dhe të gjitha links null, remove atë nyje.





Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

5.2 TRIES

- *R-way tries*
- *ternary search tries*
- *character-based operations*

Ternary search tries

- Të ruhen karakteret dhe vlerat në nyje (jo në çelësa).
- Çdo nyje ka 3 fëmijë: më të vogël (majtas), të barabartë (në mes), më të madh (djathtas).

Fast Algorithms for Sorting and Searching Strings

Jon L. Bentley* Robert Sedgewick#

Abstract

We present theoretical algorithms for sorting and searching multikey data, and derive from them practical C implementations for applications in which keys are character strings. The sorting algorithm blends Quicksort and radix sort; it is competitive with the best known C sort codes. The searching algorithm blends tries and binary search trees; it is faster than hashing and other commonly used search methods. The basic ideas behind the algorithms are old and simple, but our new implementations are novel and efficient. The symbol table routines are fully general, allow arbitrarily long strings, and have been used in commercial programs for several years. We present extensive empirical comparisons with other algorithms and implementations.

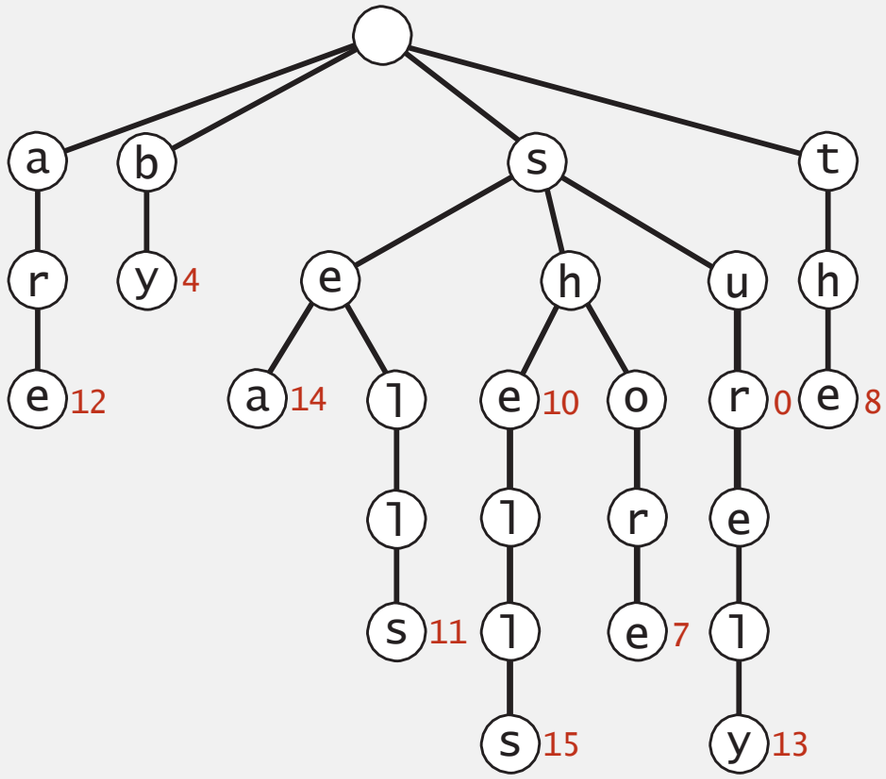
Our first program is a sorting algorithm for arrays of strings that is competitive with the most efficient string sorting programs known. The second program is a symbol table implementation that is faster than hashing, which is commonly regarded as the fastest symbol table implementation. The symbol table implementation is much more space-efficient than multiway trees, and supports more advanced searches.

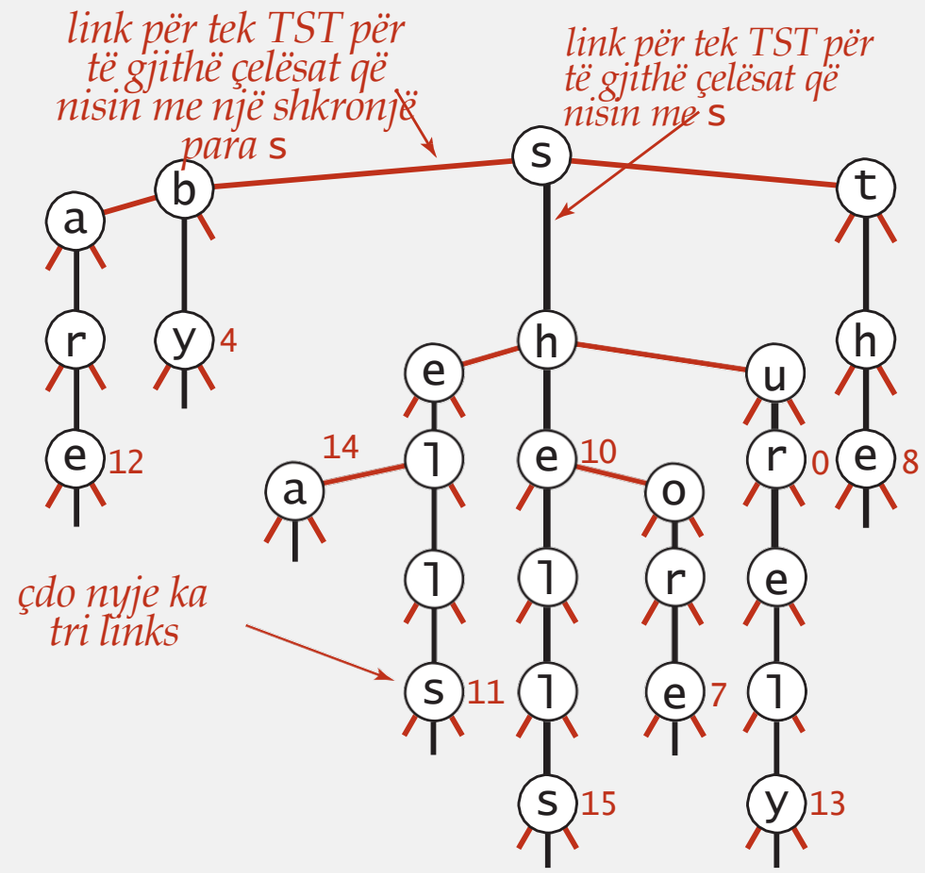
In many application programs, sorts use a Quicksort implementation based on an abstract compare operation,



Ternary search tries

- Të ruhen karakteret dhe vlerat në nyje (jo në çelësa).
- Çdo nyje ka 3 fëmijë: më të vogël (majtas), të barabartë (në mes), më të madh (djathtas).

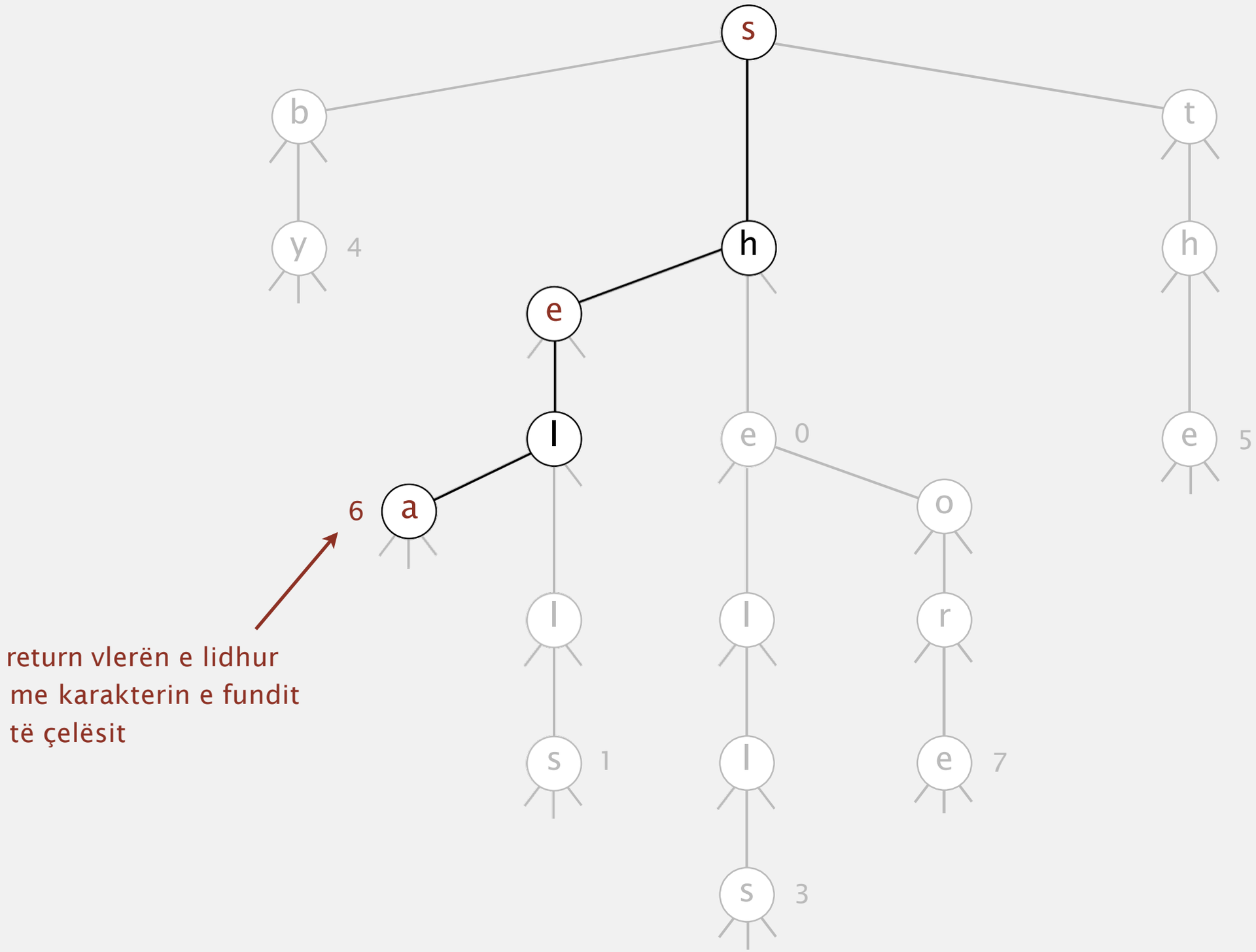




TST paraqitja e një trie

Search hit në një TST

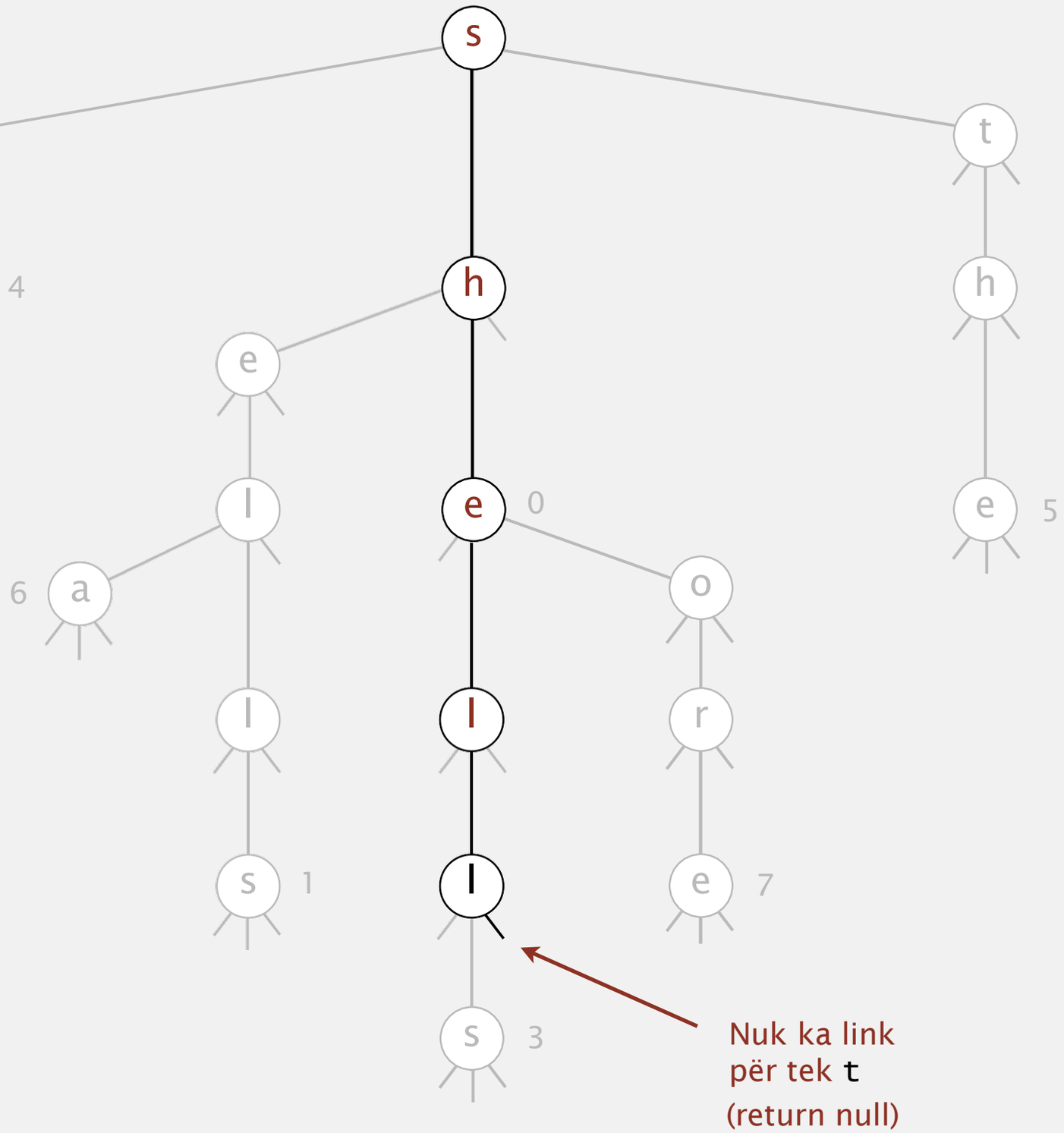
get("sea")



return vlerën e lidhur
me karakterin e fundit
të çelësit

Search hit në një TST

get("shelter")



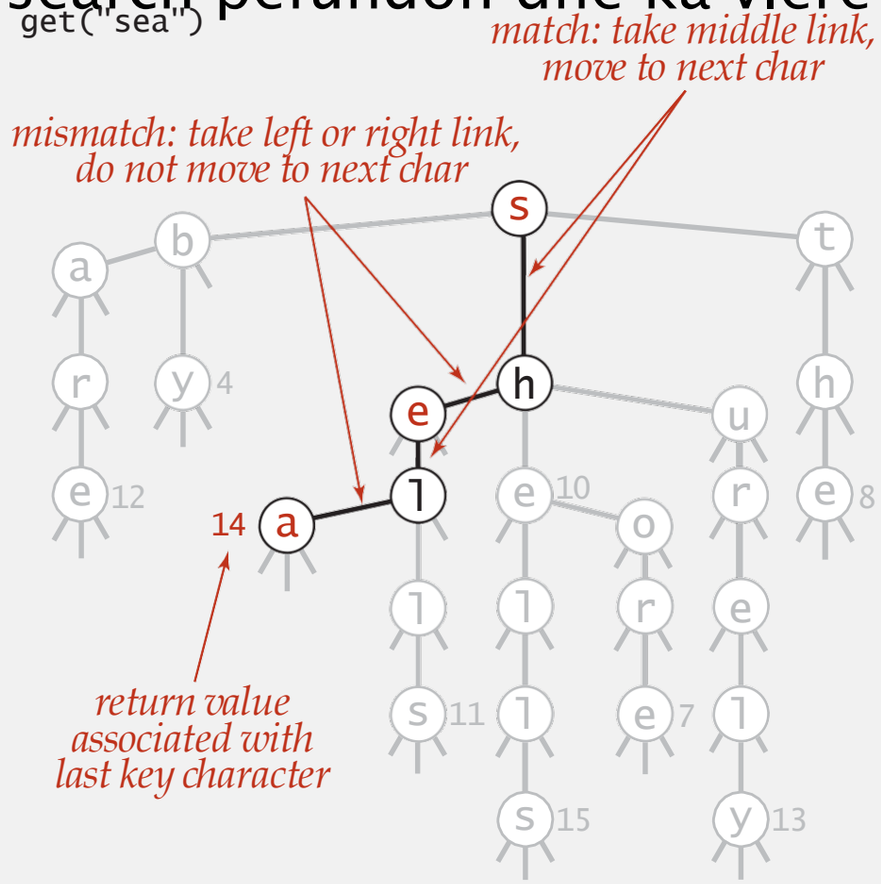
Search në një TST

Të përcillen links përkatës tek çdo karakter në çelës.

- Nëse vlera më e vogël, link majtas; nëse vlera më e madhe, link djathtas.
- Nëse vlera e barabartë, link i mesëm dhe vazhdo tek karakteri i ardhshëm në çelës.

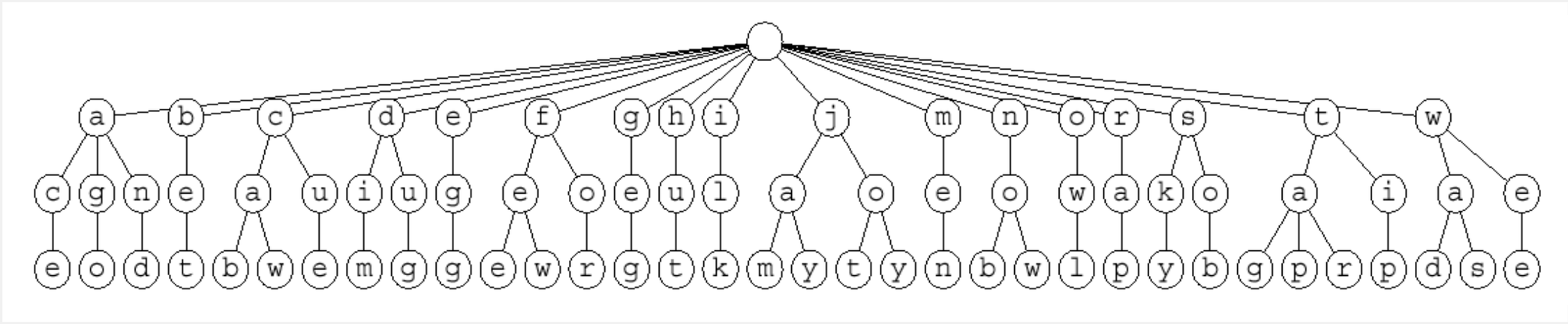
Search hit. Nyja ku search përfundon dhe ka vlerë jo-null.

Search miss. Nyja ku search përfundon dhe ka vlerë null ose përfundon në null link



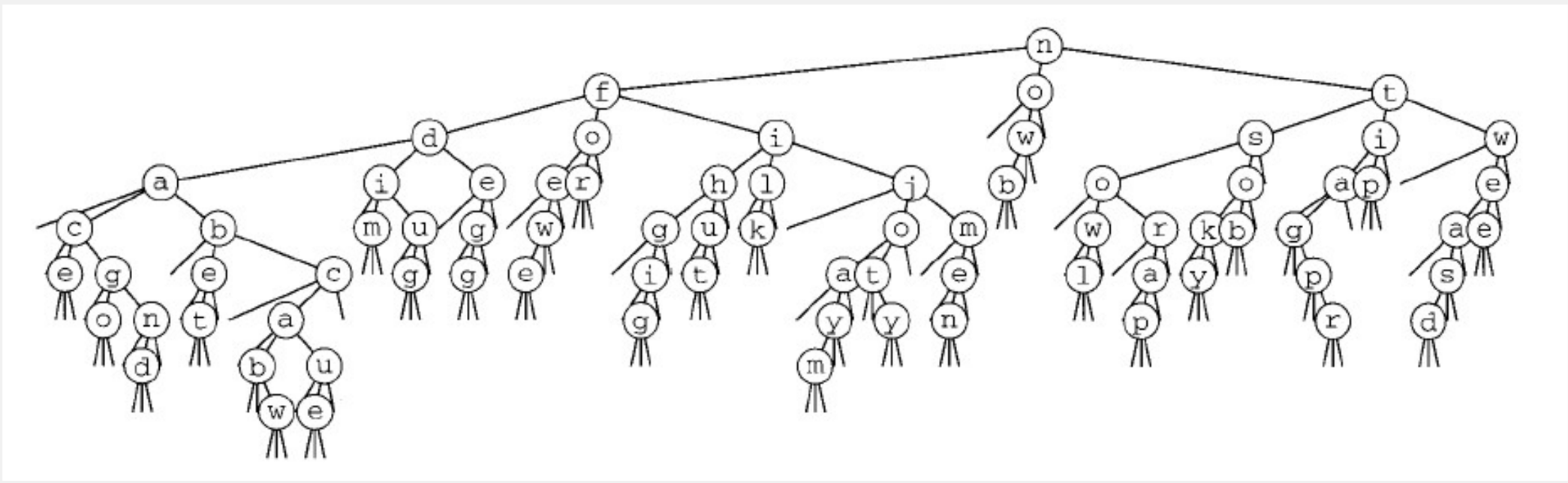
26-way trie vs. TST

26-way trie. 26 null links në çdo leaf.



26- way trie (1035 null links, nuk janë të paraqitur)

TST. 3 null links në çdo leaf.



TST (155 null links)

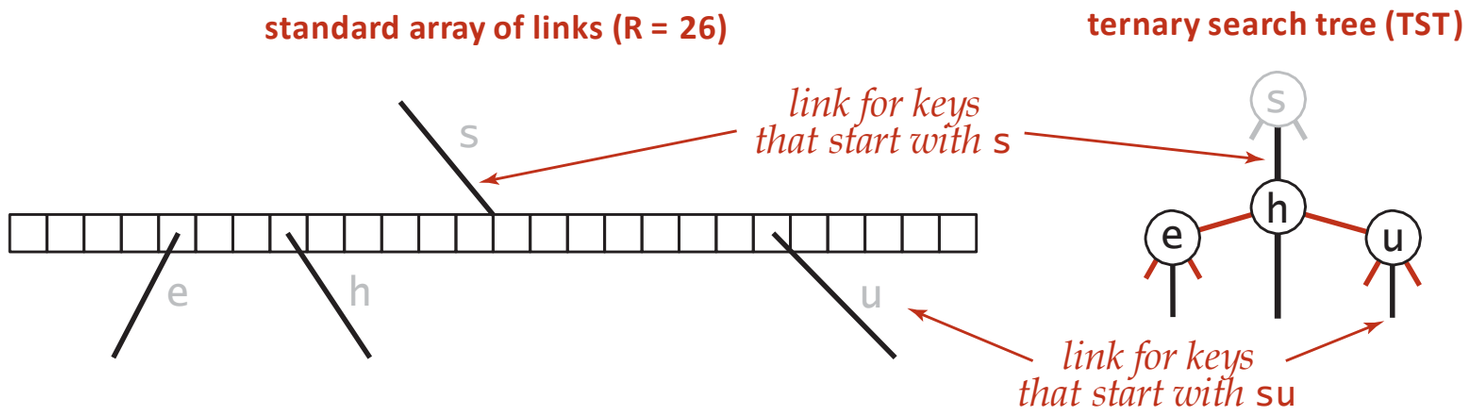
now
for
tip
ilk
dim
tag
jot
sob
nob
sky
hut
ace
bet
men
egg
few
jay
owl
joy
rap
gig
wee
was
cab
wad
caw
cue
fee
tap
ago
tar
jam
dug
and

TST - paraqitja në Java

Një TST nyje ka pesë fusha:

- Një vlerë.
- Një karakter c.
- Një referencë tek një TST majtas.
- Një reference tek një TST në mes.
- Një reference tek një TST djathtas.

```
private class Node
{
   private Value val;
   private char c;
   private Node left, mid, right;
}
```



Trie node representations

TST: Java implementation

```
public class TST<Value>
{
   private Node root;

   private class Node
   {  /* see previous slide */  }

   public void put(String key, Value val)
   {  root = put(root, key, val, 0);  }

   private Node put(Node x, String key, Value val, int d)
   {
      char c = key.charAt(d);
      if (x == null) {  x = new Node(); x.c = c;  }
      if      (c < x.c)              x.left  = put(x.left,  key, val, d);
      else if (c > x.c)              x.right = put(x.right, key, val, d);
      else if (d < key.length() - 1) x.mid   = put(x.mid,   key, val, d+1);
      else                           x.val   = val;
      return x;
   }

   ...
```

TST:Java implementation

```
   ⋮
public boolean contains(String key)
{  return get(key) != null;  }

public Value get(String key)
{
   Node x = get(root, key, 0);
   if (x == null) return null;
   return x.val;
}

   private Node get(Node x, String key, int d)
   {
      if (x == null) return null;
      char c = key.charAt(d);
      if      (c < x.c)              return get(x.left,  key, d);
      else if (c > x.c)              return get(x.right, key, d);
      else if (d < key.length() - 1) return get(x.mid,   key, d+1);
      else                           return x;
   }
}
```

TST vs. hashing

Hashing.

- Duhet të kontrollohet i gjithë çelësi.
- Search hits dhe misses kushtojnë përafërsisht njejtë
- Performanca mbështetet tek hash funkcioni.
- Nuk përkrahë symbol table operacione të rradhitura.

TST-të.

- Punon vetëm me string (ose digjital) çelësa.
- Kontrollon saktësisht aq sa duhet karaktere të çelësit.
- Search miss mund të ketë vetëm disa karaktere.
- Përkrahë symbol table operacione të rradhitura.

Përfundim. TST-të janë:

- më të shpejta sesa hashing (veçanërisht për search misses).
- më fleksibil sesa red-black BSTtë.